

REFLECTIVE DLL INJECTION

31기 육은서

Contents.

01

Reflective Dll Injection

02

Reflective Dll Injection 과정

- inject.exe
- reflectiveloader 호출
- dll 메모리에 매핑
- IAT 재작성
- Relocation
- Entrypoint 호출 및 진입

03

향후 칼럼 방향

-마킹제이

04

QnA

Reflective DLL INJECTION

DLL INJECTION

탐지가 쉬움

프로세스 메모리에 DLL 데이터 삽입 후 매핑하여
백그라운드에서 탐지되지 않는
Reflective DLL Injection

Reflective DLL INJECTION

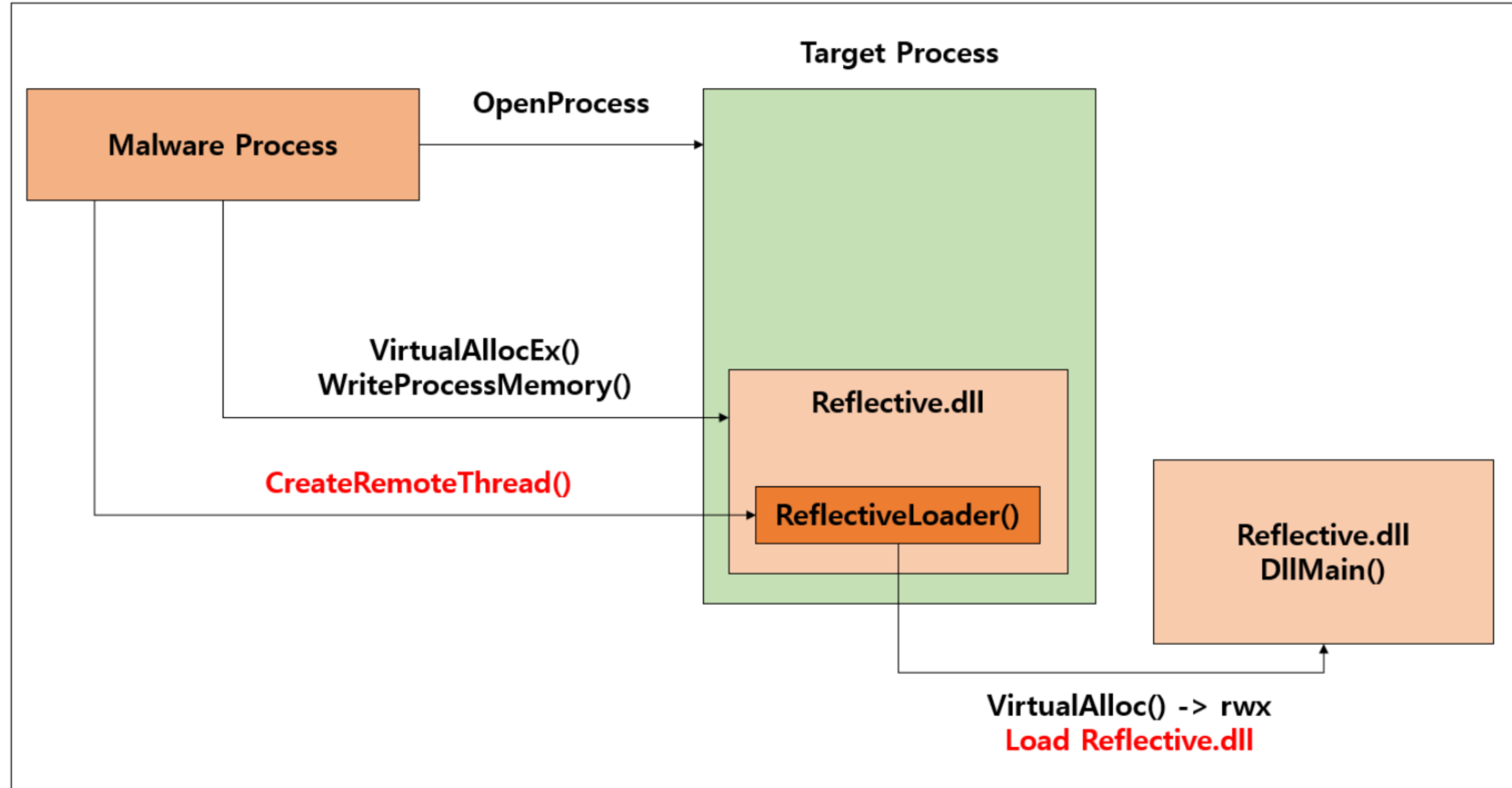
DLL INJECTION

탐지가 쉬움

프로세스 메모리에 DLL 데이터 삽입 후 매핑하여
백그라운드에서 탐지되지 않는
Reflective DLL Injection

1. 프로세스의 핸들 구하기
2. 프로세스의 메모리 공간 확보하기
3. ReflectiveLoader 호출
4. DLL 메모리에 매핑(헤더와 섹션 메모리에 로드시킴)
5. Import Table 재빌딩(DLL 재구성 및 삽입)
6. Relocation
7. DLL 동작준비 완료

Reflective DLL INJECTION



inject.exe

```
00007FF6F8711070 BA 00000080 mov edx,80000080
00007FF6F8711075 49:88CF mov rcx,r15
00007FF6F8711078 C74424 28 80000000 mov dword ptr ss:[rsp+28],80
00007FF6F8711080 C74424 20 03000000 mov dword ptr ss:[rsp+20],3
00007FF6F8711088 FF15 02900000 call qword ptr ds:[<CreateFileA>]
00007FF6F871108E 4C:88F0 mov r14,rcx
00007FF6F8711091 48:83F8 FF cmp rax,FFFFFFFFFFFFFFFF
00007FF6F8711095 75 1E jne inject.x64.7FF6F87110B5
00007FF6F8711097 FF15 AB8F0000 call qword ptr ds:[<GetLastError>]
00007FF6F871109D 48:8D15 04E30000 lea rdx,qword ptr ds:[7FF6F871F3A8]
00007FF6F87110A4 48:8D0D 1DE30000 lea rcx,qword ptr ds:[7FF6F871F3C8]
00007FF6F87110A8 44:8BC0 mov r8d,eax
00007FF6F87110AE E8 51050000 call inject.x64.7FF6F8711604
00007FF6F87110B3 E8 61 jmp inject.x64.7FF6F8711116
00007FF6F87110B5 33D2 xor edx,edx
00007FF6F87110B7 48:8BC8 mov rcx,rcx
00007FF6F87110BA FF15 C08F0000 call qword ptr ds:[<GetFileSize>]
00007FF6F87110C0 8BF0 mov esi,eax
00007FF6F87110C2 8D46 FF lea eax,qword ptr ds:[rsi-1]
00007FF6F87110C5 83F8 FD cmp eax,FFFFFFFF
00007FF6F87110C8 0F87 CB010000 ja inject.x64.7FF6F8711299
00007FF6F87110CE 4C:89A424 A0000000 mov qword ptr ss:[rsp+A0],r12
00007FF6F87110D6 FF15 548F0000 call qword ptr ds:[<GetProcessHeap>]
00007FF6F87110DC 44:8BC6 mov r8d,esi
00007FF6F87110DF 48:8BC8 mov rcx,rcx
00007FF6F87110E2 33D2 xor edx,edx
00007FF6F87110E4 FF15 8E8F0000 call qword ptr ds:[<HeapAlloc>]
00007FF6F87110EA 4C:8BE0 mov r12,rcx
00007FF6F87110ED 48:85C0 test rax,rax
00007FF6F87110F0 75 47 jne inject.x64.7FF6F8711139
00007FF6F87110F2 FF15 508F0000 call qword ptr ds:[<GetLastError>]
```

주소	Hex	ASCII
0000023E0018D000	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D010	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D020	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D030	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D040	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D050	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D060	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D070	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D080	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø
0000023E0018D090	0D F0 AD BA 0D F0 AD BA 0D F0 AD BA 0D F0 AD BA	.ø.ø.ø.ø.ø.ø.ø.ø

```
lea r9,qword ptr ss:[rsp+48]
mov r8d,esi
mov rdx,rax
mov rcx,r14
mov qword ptr ss:[rsp+20],rdi
call qword ptr ds:[<ReadFile>]
```

dll 파일 핸들과 할당받은 heap에
dll 파일 데이터 입력

주소	Hex	ASCII
0000023E0018D000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....yy
0000023E0018D010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	@.
0000023E0018D020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0000023E0018D030	00 00 00 00 00 00 00 00 00 00 00 00 F8 00 00 00	ø.
0000023E0018D040	0E 1F BA 0E 00 84 09 CD 21 B8 01 4C CD 21 54 68	..ø...i!..Li!
0000023E0018D050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program can
0000023E0018D060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DO
0000023E0018D070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode...\$.....
0000023E0018D080	E4 E9 59 8A A0 88 37 D9 A0 88 37 D9 A0 88 37 D9	æéY. .7Ù .7Ù .
0000023E0018D090	51 4E F8 D9 82 88 37 D9 51 4E F9 D9 F4 88 37 D9	QNøÙ=.7ÙQNøÙø.

inject.exe

●	00007FF6F87111E6	44:8BC5	mov r8d,ebp
●	00007FF6F87111E9	33D2	xor edx,edx
●	00007FF6F87111EB	B9 3A040000	mov ecx,43A
RIP → ●	00007FF6F87111F0	FF15 428E0000	call qword ptr ds:[<OpenProcess>]

●	00007FF6F87111BC	48:8B4C24 40	mov rcx,qword ptr ss:[rsp+40]
●	00007FF6F87111C1	4C:8D4424 50	lea r8,qword ptr ss:[rsp+50]
●	00007FF6F87111C6	45:33C9	xor r9d,r9d
●	00007FF6F87111C9	33D2	xor edx,edx
●	00007FF6F87111CB	48:897C24 28	mov qword ptr ss:[rsp+28],rdi
●	00007FF6F87111D0	48:897C24 20	mov qword ptr ss:[rsp+20],rdi
●	00007FF6F87111D5	FF15 358E0000	call qword ptr ds:[<AdjustTokenPrivilege>]
●	00007FF6F87111DB	48:8B4C24 40	mov rcx,qword ptr ss:[rsp+40]
●	00007FF6F87111E0	FF15 A28E0000	call qword ptr ds:[<CloseHandle>]
●	00007FF6F87111E6	44:8BC5	mov r8d,ebp
●	00007FF6F87111E9	33D2	xor edx,edx
●	00007FF6F87111EB	B9 3A040000	mov ecx,43A
●	00007FF6F87111F0	FF15 428E0000	call qword ptr ds:[<OpenProcess>]
RIP → ●	00007FF6F87111F6	48:8BF8	mov rdi,rax

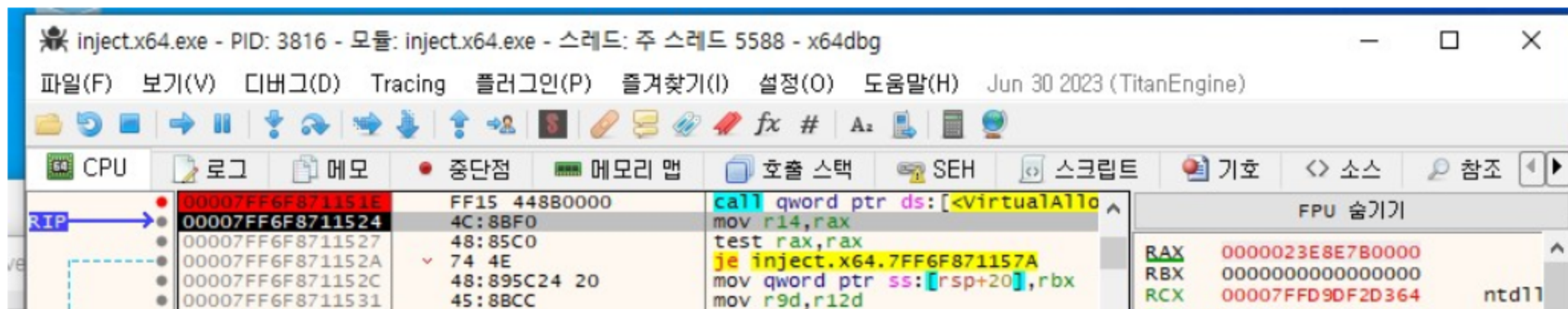
FPU 슬기기		
RAX	00000000000000D4	'0'
RBX	0000023E001B5590	&"C:\
RCX	00007FFD9DF2D524	ntdll
RDY	0000000000000000	
RBP	00000000000000E0	L'I'
RSP	000000DEBBAFF680	
RSI	00000000000010400	
RDI	0000000000000000	
R8	000000DEBBAFF608	
R9	00000000000000E0	L'I'

0000000000000000D4

그림판 핸들

"mspaint.exe" 오픈하고 핸들 갓

inject.exe



inject.x64.exe - PID: 3816 - 모듈: inject.x64.exe - 스레드: 주 스레드 5588 - x64dbg

파일(F) 보기(V) 디버그(D) Tracing 플러그인(P) 즐겨찾기(I) 설정(O) 도움말(H) Jun 30 2023 (TitanEngine)

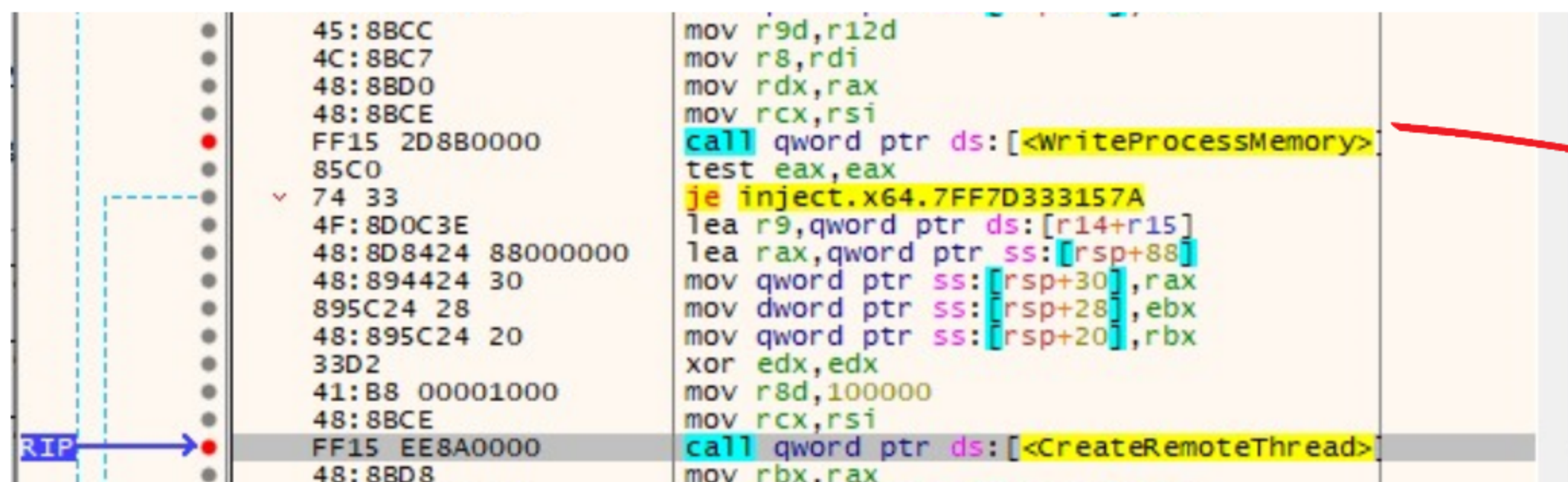
CPU 로그 메모 중단점 메모리 맵 호출 스택 SEH 스크립트 기호 소스 참조

RIP → 00007FF6F871151E FF15 448B0000 call qword ptr ds:[<VirtualAlloc>]
00007FF6F8711524 4C:88F0 mov r14,rax
00007FF6F8711527 48:85C0 test rax,rax
00007FF6F871152A 74 4E je inject.x64.7FF6F871157A
00007FF6F871152C 48:895C24 20 mov qword ptr ss:[rsp+20],rbx
00007FF6F8711531 45:8BCC mov r9d,r12d

FPU 숨기기

RAX	0000023E8E7B0000
RBX	0000000000000000
RCX	00007FFD9DF2D364 ntdll

0000023E8E6A0000	0000000000000000	사	자	Heap (ID 0)		PRV	
0000023E8E6C1000	000000000000DF00	사	자	예약됨 (0000023E8E6A0000)		PRV	
0000023E8E7A0000	0000000000000100	사	자			MAP	-R---
0000023E8E7B0000	0000000000001100	사	자			PRV	ERW--
0000023E8E880000	0000000000000500	사	자	Heap (ID 2)		PRV	-RW--
0000023E8E885000	0000000000000800	사	자	예약됨 (0000023E8E880000)		PRV	
00007FF6F8711531	0000000000000500	사	자			MAP	-R---



45:8BCC mov r9d,r12d
4C:8BC7 mov r8,rdi
48:8BD0 mov rdx,rax
48:8BCE mov rcx,rsi
FF15 2D8B0000 call qword ptr ds:[<WriteProcessMemory>]
85C0 test eax,eax
74 33 je inject.x64.7FF7D333157A
4F:8D0C3E lea r9,qword ptr ds:[r14+r15]
48:8D8424 88000000 lea rax,qword ptr ss:[rsp+88]
48:894424 30 mov qword ptr ss:[rsp+30],rax
895C24 28 mov dword ptr ss:[rsp+28],ebx
48:895C24 20 mov qword ptr ss:[rsp+20],rbx
33D2 xor edx,edx
41:B8 00001000 mov r8d,100000
48:8BCE mov rcx,rsi
FF15 EE8A0000 call qword ptr ds:[<CreateRemoteThread>]
48:8BD8 mov rbx,rax

reflective.dll 데이터 작성

ReflectiveLoader 호출

호출을 위한 필요함수 구하기

LoadLibraryA(), VirtualAlloc(), GetProcAddress(), FlushInstructionCache()

kernel32.dll

ntdll.dll

```
169 // loop while we still have imports to find
170 while( usCounter > 0 )
171 {
172     // compute the hash values for this function name
173     dwHashValue = hash( (char *) ( uiBaseAddress + DEREF_32( uiNameArray ) ) );
174
175     // if we have found a function we want we get its virtual address
176     if( dwHashValue == LOADLIBRARYA_HASH || dwHashValue == GETPROCADDRESS_HASH || dwHashValue == VIRTUALALLOC_HASH )
177     {
178         // get the VA for the array of addresses
179         uiAddressArray = ( uiBaseAddress + ((PIMAGE_EXPORT_DIRECTORY) uiExportDir->AddressOfFunctions ) );
180
181         // use this functions name ordinal as an index into the array of name pointers
182         uiAddressArray += ( DEREF_16( uiNameOrdinals ) * sizeof(DWORD) );
183
184         // store this functions VA
185         if( dwHashValue == LOADLIBRARYA_HASH )
186             pLoadLibraryA = (LOADLIBRARYA)( uiBaseAddress + DEREF_32( uiAddressArray ) );
187         else if( dwHashValue == GETPROCADDRESS_HASH )
188             pGetProcAddress = (GETPROCADDRESS)( uiBaseAddress + DEREF_32( uiAddressArray ) );
189         else if( dwHashValue == VIRTUALALLOC_HASH )
190             pVirtualAlloc = (VIRTUALALLOC)( uiBaseAddress + DEREF_32( uiAddressArray ) );
191
192         // decrement our counter
```

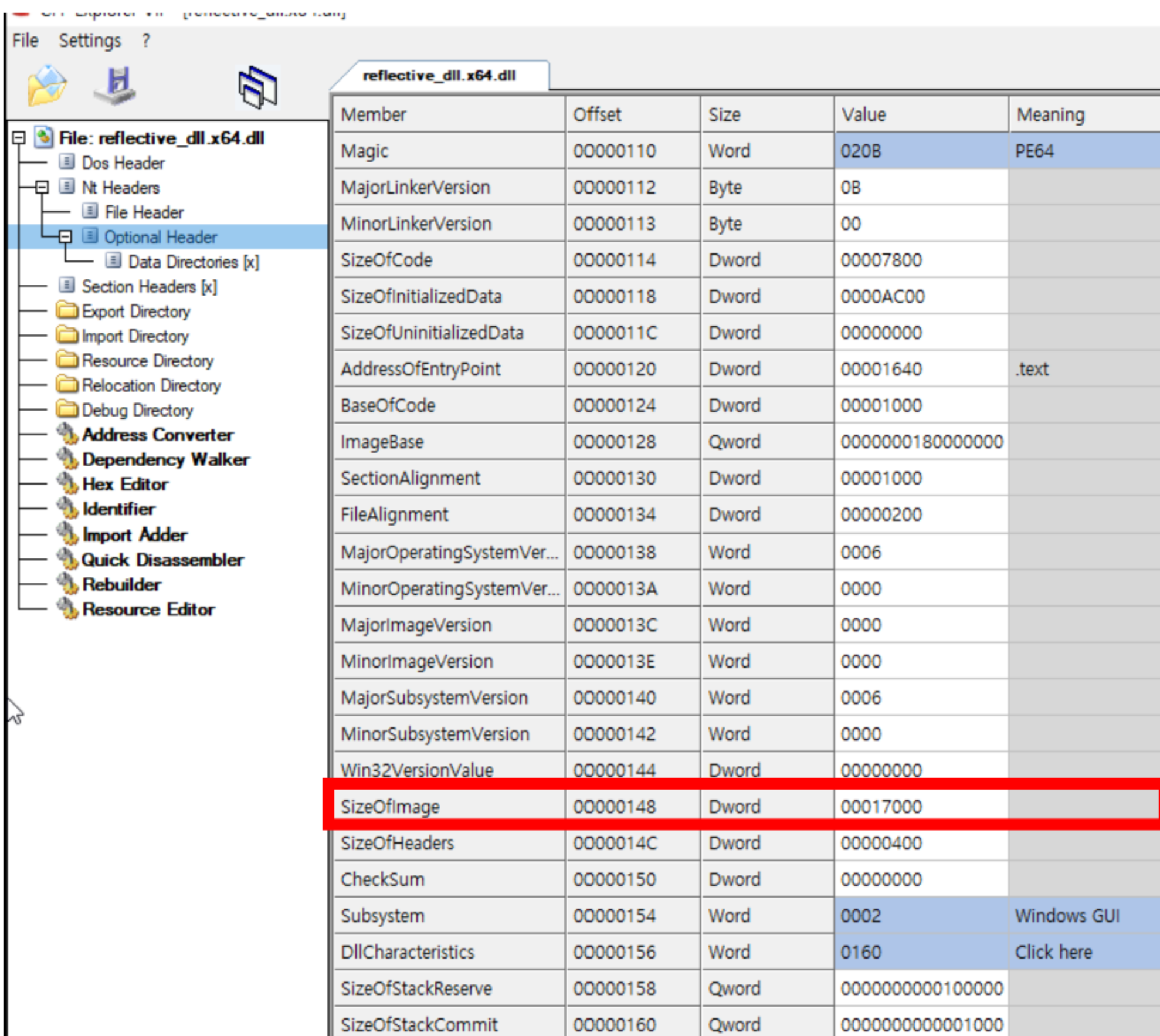
현재 주소의 해시를 저장하여
구하는 함수와 해시가 같다면
주소 저장

ReflectiveLoader 호출

	offset	< File >	<Memory>	address
Member	00000000	DOS header	DOS header	01000000
Magic	00000040	DOS stub	DOS stub	01000040
MajorLinkerVersion	000000E0	NT header	NT header	010000E0
MinorLinkerVersion	000001D8	Section header(".text")	Section header(".text")	010001D8
SizeOfCode	00000200	Section header(".data")	Section header(".data")	01000200
SizeOfInitializedData	00000228	Section header(".rsrc")	Section header(".rsrc")	01000228
BaseOfCode	00000400	Null	Null	
SectionAlignment	00000400			01001000
FileAlignment		Section(".text")	7748 Section(".text")	
MajorOperatingSystemVersion		Null	Null	
MinorOperatingSystemVersion	00007C00	Section(".data")	Null	
MajorSubsystemVersion	800	Null	1BA8 Section(".data")	01009000
MinorSubsystemVersion	00008400	Section(".rsrc")	Null	
Win32VersionValue		Null	8314 Section(".rsrc")	0100B000
SizeOfImage				
SizeOfHeaders	8400			
Checksum				
Subsystem	00010800			
DllCharacteristics	00000156			
SizeOfStackReserve				
SizeOfStackCommit	00000160			

PE파일
윈도우 실행파일
DLL, EXE 등 종류가 다양하다
파일이 실행되기 위한 모든 정보
총집합

ReflectiveLoader 호출



Member	Offset	Size	Value	Meaning
Magic	00000110	Word	020B	PE64
MajorLinkerVersion	00000112	Byte	0B	
MinorLinkerVersion	00000113	Byte	00	
SizeOfCode	00000114	Dword	00007800	
SizeOfInitializedData	00000118	Dword	0000AC00	
SizeOfUninitializedData	0000011C	Dword	00000000	
AddressOfEntryPoint	00000120	Dword	00001640	.text
BaseOfCode	00000124	Dword	00001000	
ImageBase	00000128	Qword	0000000180000000	
SectionAlignment	00000130	Dword	00001000	
FileAlignment	00000134	Dword	00000200	
MajorOperatingSystemVersion	00000138	Word	0006	
MinorOperatingSystemVersion	0000013A	Word	0000	
MajorImageVersion	0000013C	Word	0000	
MinorImageVersion	0000013E	Word	0000	
MajorSubsystemVersion	00000140	Word	0006	
MinorSubsystemVersion	00000142	Word	0000	
Win32VersionValue	00000144	Dword	00000000	
SizeOfImage	00000148	Dword	00017000	
SizeOfHeaders	0000014C	Dword	00000400	
Checksum	00000150	Dword	00000000	
Subsystem	00000154	Word	0002	Windows GUI
DllCharacteristics	00000156	Word	0160	Click here
SizeOfStackReserve	00000158	Qword	0000000000100000	
SizeOfStackCommit	00000160	Qword	0000000000001000	

DLL 파일의 이미지 크기만큼
메모리 할당

이 공간에 dll 헤더 데이터 작성

ReflectiveLoader 호출

CFF Explorer VII - [reflective_dll.x64.dll]

File Settings ?

reflective_dll.x64.dll

Member	Offset	Size	Value	Meaning
Machine	000000FC	Word	8664	AMD64 (K8)
NumberOfSections	000000FE	Word	0006	
TimeStamp	00000100	Dword	52283BF2	
PointerToSymbolT...	00000104	Dword	00000000	
NumberOfSymbols	00000108	Dword	00000000	
SizeOfOptionalHea...	0000010C	Word	00F0	
Characteristics	0000010E	Word	2022	Click here

섹션 수

Optional header 크기

CFF Explorer VII - [reflective_dll.x64.dll]

File Settings ?

reflective_dll.x64.dll

Member	Offset	Size	Value	Meaning
Magic	00000110	Word	020B	PE64
MajorLinkerVersion	00000112	Byte	08	
MinorLinkerVersion	00000113	Byte	00	
SizeOfCode	00000114	Dword	00007800	
SizeOfInitializedData	00000118	Dword	0000AC00	
SizeOfUninitializedData	0000011C	Dword	00000000	
AddressOfEntryPoint	00000120	Dword	00001640	.text
BaseOfCode	00000124	Dword	00001000	

BaseOfcode 구하고
섹션 데이터 입력

IAT 작성

IAT란? Import Address Table -> 사용되는 함수들 정보가 담겨 있음

앞에서 로드된 파일로 IAT 작성

CFF Explorer VII - [reflective_dll.x64.dll]

File Settings ?

reflective_dll.x64.dll

Member	Offset	Size	Value	Section
Export Directory RVA	00000180	Dword	0000F190	.rdata
Export Directory Size	00000184	Dword	00000067	
Import Directory RVA	00000188	Dword	0000EA60	.rdata
Import Directory Size	0000018C	Dword	0000003C	
Resource Directory RVA	00000190	Dword	00015000	.rsrc

import directory

Name 주소 구해서
USER32.dll 안
MessageBoxA() 함수 주소 겠

CFF Explorer VII - [reflective_dll.x64.dll]

File Settings ?

reflective_dll.x64.dll

Module Name	Imports	OFTs	TimeStamp	ForwarderChain	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
USER32.dll	1	0000ECA8	00000000	00000000	0000ECC6	00009208
KERNEL32.dll	64	0000EAA0	00000000	00000000	0000F17A	00009000

Relocation

이미지가 충돌 없이 메모리에 로드되기 위함
=다른 프로세스가 이미 자리를 차지했다면
재배치를 이용하여 다른 주소 공간으로 로딩

CFF Explorer VII - [reflective_dll.x64.dll]

File Settings ?

reflective_dll.x64.dll

File: reflective_dll.x64.dll

- Dos Header
- Nt Headers
 - File Header
 - Optional Header
 - Data Directories [x]
- Section Headers [x]
- Export Directory
- Import Directory
- Resource Directory
- Relocation Directory
- Debug Directory
- Address Converter
- Dependency Walker
- Hex Editor
- Identifier
- Import Adder
- Quick Disassembler
- Rebuilder
- Resource Editor

Member	Offset	Size	Value	Section
Export Directory RVA	00000180	Dword	0000F190	.rdata
Export Directory Size	00000184	Dword	00000067	
Import Directory RVA	00000188	Dword	0000EA60	.rdata
Import Directory Size	0000018C	Dword	0000003C	
Resource Directory RVA	00000190	Dword	00015000	.rsrc
Resource Directory Size	00000194	Dword	000001E0	
Exception Directory RVA	00000198	Dword	00014000	.pdata
Exception Directory Size	0000019C	Dword	0000078C	
Security Directory RVA	000001A0	Dword	00000000	
Security Directory Size	000001A4	Dword	00000000	
Relocation Directory RVA	000001A8	Dword	00016000	.reloc
Relocation Directory Size	000001AC	Dword	00000524	
Debug Directory RVA	000001B0	Dword	00009290	.rdata
Debug Directory Size	000001B4	Dword	00000038	

CFF Explorer VII - [reflective_dll.x64.dll]

File Settings ?

reflective_dll.x64.dll

RVA	Size Of Block	Items
Dword	Dword	N/A
00009000	0000004C	34
0000B000	000000C8	96
0000C000	00000208	256
0000D000	000000DC	106
0000E000	00000010	4
00010000	000000E8	112
00011000	00000034	22

↓
각 item에
AllocAddress - ImageBase(reflective.dll)

EntryPoint 호출 및 진입

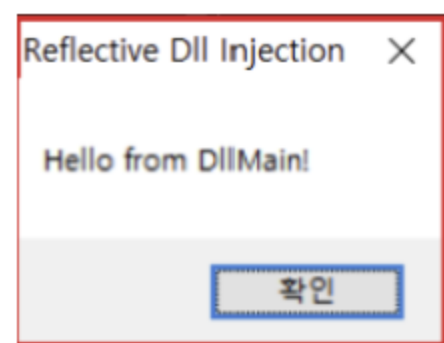
dll 파일의 EntryPoint값 가져옴

할당받은 Entrypoint 시작주소+EntryPoint값=EntryPoint주소

NtFlushInstructionCache()

EntryPoint주소로 이동

DllMain()



Member	Offset	Size	Value	Meaning
Magic	00000110	Word	020B	PE64
MajorLinkerVersion	00000112	Byte	0B	
MinorLinkerVersion	00000113	Byte	00	
SizeOfCode	00000114	Dword	00007800	
SizeOfInitializedData	00000118	Dword	0000AC00	
SizeOfUninitializedData	0000011C	Dword	00000000	
AddressOfEntryPoint	00000120	Dword	00001640	.text
BaseOfCode	00000124	Dword	00001000	
ImageBase	00000128	Qword	0000000180000000	
SectionAlignment	00000130	Dword	00001000	
FileAlignment	00000134	Dword	00000200	

향후 칼럼 방향

Home > 전체기사

새로운 프로세스 주입 공격 기법 마킹제이, 기존 EDR 농락해

좋아요 14개

| 입력: 2023-06-28 12:22



가 가



HIWARE 통합 접근 및 계정 권한 관리 솔루션

접근통제 | 권한관리 | 계정관리 | 인증강화 | 로그감사 **ND NETAND**

새로운 공격 기법...기존 프로세스 주입 공격법과는 사뭇 달라

요약 : 보안 외신 해커뉴스에 의하면 보안 전문가들이 악성 프로세스를 주입하는 새로운 전략을 개발했다고 한다. 이 기법에는 마킹제이(MockingJay)라는 이름이 붙었다. 이 전략을 통해 공격자들은 보안 장치들을 우회하여 악성 코드를 실행시킬 수 있게 된다. 보안 업체 시큐리티조스(Security Joes)의 연구원들이 개발한 방법이다. 공격을 성공시키기 위해서는 취약한 DLL 하나가 필요하며 코드를 올바른 위치에 복사해 넣어야 한다고 한다.

마킹제이

시큐리티조스가 EDR를 피하는 프로세스 인젝션 공격법을 개발했다.

기사에는 메모리 구역 할당이나, 권한 조작, 스레드 제작이 필요없다는데 어떤 공격 기법인걸까?



QnA

yes0823bs@naver.com